DVWA DOM Based XSS 渗透测试训练报告

一、测试概要

项目	内容
漏洞类型	DOM 型跨站脚本攻击 (DOM Based XSS)
训练平台	DVWA (Damn Vulnerable Web Application)
测试模块	XSS (DOM)
安全等级	Low / Medium / High
测试日期	2026-04-27
测试人员	Mr.X
报告编号	DVWA-XSS-D-2026-003

二、售前业务风险分析

本章节面向非技术人员，以业务视角阐述该类漏洞对企业的实际影响。

2.1 漏洞概述

DOM 型 XSS 是一种完全发生在**客户端浏览器**中的跨站脚本攻击。恶意脚本不经过服务器处理，而是由页面中的 JavaScript 代码直接从 URL 中读取用户输入并写入页面 DOM，导致恶意代码执行。由于攻击流量不经过服务器，**传统 WAF 和服务端日志均无法检测到此类攻击**。

2.2 业务影响分析

影响维度	风险描述	严重程度
用户数据安全	攻击者可窃取用户 Cookie/Session，接管账户	严重
检测盲区	攻击不经过服务器，WAF 和服务端日志**完全无感知**	**严重**
品牌信誉	页面被篡改、钓鱼弹窗等，用户不知情	高
合规风险	用户数据被窃取，违反《网络安全法》《个人信息保护法》	高
溯源困难	服务端无攻击痕迹，事后取证难度极大	高

2.3 三种 XSS 类型风险对比

维度	反射型 XSS	存储型 XSS	DOM 型 XSS
攻击载荷位置	服务端响应	数据库	**浏览器 DOM**
经过服务器	是	是	**否**
WAF 可检测	是	是	**否**
服务端日志	有记录	有记录	**无记录**
触发条件	点击恶意链接	访问页面	点击恶意链接
隐蔽性	中	低	**高**

2.4 典型攻击场景

1. **无痕钓鱼**: 攻击者构造恶意 URL，整个攻击在浏览器端完成，服务端无任何日志记录
2. **绕过 WAF**: 企业部署了 Web 应用防火墙也无法拦截，因为恶意载荷不经过网络传输到服务端
3. **URL 片段攻击**: 利用#锚点注入恶意脚本，#后的内容不会发送到服务器，实现完全隐蔽攻击
4. **会话劫持**: 窃取 Cookie 后以用户身份登录，且无法通过服务端审计发现异常

三、测试环境

项目	说明
靶机平台	DVWA (Damn Vulnerable Web Application)
测试模块路径	/vulnerabilities/xss_d/
请求方式	GET
注入参数	default
功能描述	语言选择下拉框（English / French / Spanish / German）
浏览器	Chrome / Firefox
辅助工具	浏览器开发者工具 (F12)

前端核心代码（漏洞根源）

三个等级共用以下前端 JavaScript，这是 DOM-XSS 的**漏洞根源**：

```
if (document.location.href.indexOf("default=") >= 0) {
var lang = document.location.href.substring(
document.location.href.indexOf("default=") + 8
);
document.write("<option value='" + lang + "'>" + decodeURI(lang) + "</option>");
}
```

漏洞分析:

- 从 document.location.href 中直接提取 default=后的内容
- 使用 document.write()将提取内容**未经任何过滤**写入 DOM
- decodeURI()会将 URL 编码还原，使%3Cscript%3E 变回<script>
- 三个等级的差异仅在于**服务端 PHP 对请求的预检**，前端漏洞代码始终存在

四、Low 等级测试

4.1 源码分析

服务端 (PHP):

```
<?php
# No protections, anything goes
?>
```

前端 (JavaScript):见上文核心代码。

安全措施	分析
服务端过滤	无— 注释明确写了 "No protections"
前端过滤	无—document.write()直接输出
输出编码	无

结论: 服务端和前端均无任何防护，攻击者可直接注入任意脚本。

4.2 测试步骤

Step 1: 在 DVWA 中将 Security Level 设置为 Low

Step 2: 访问 XSS (DOM) 模块，确认正常功能

URL: http://<target>/vulnerabilities/xss_d/?default=English
页面: 下拉框显示 English 为选中项

Step 3: 注入基础 XSS Payload

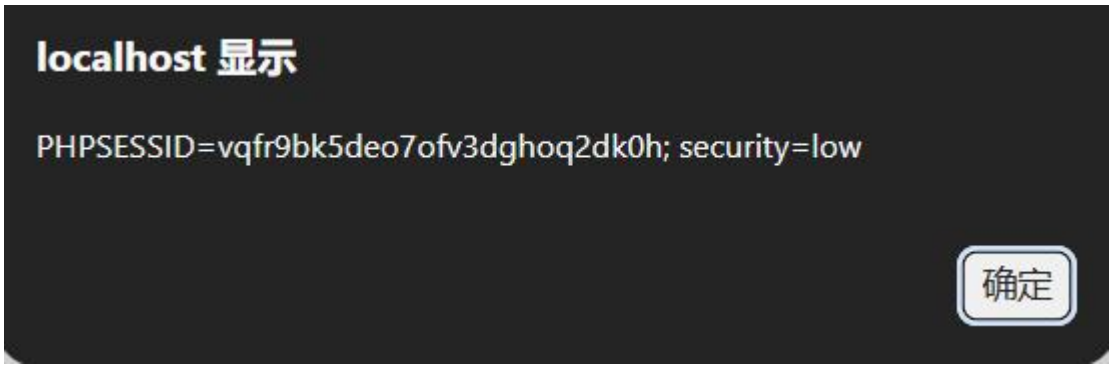
?default=<script>alert('XSS')</script>

预期结果: 浏览器弹窗显示 "XSS"。JavaScript 从 URL 读取<script>alert('XSS')</script>并通过 document.write()写入页面，浏览器解析执行。

Step 4: 窃取 Cookie Payload

?default=<script>alert(document.cookie)</script>

预期结果: 弹窗显示当前用户 Cookie（含 PHPSESSID、security 等字段）。



Step 5: 查看页面源码（F12），确认攻击发生在 DOM 层

打开开发者工具 → Elements 面板，观察<select>中被注入了<script>标签。注意：查看 "View Page Source" 看不到注入内容（因为恶意代码是 JavaScript 动态写入的，不在原始 HTML 中）。

```
<!DOCTYPE html>
<html lang="en-GB"> view-source scroll
  <script id="allow-copy_script" src="chrome-extension://kkemgiffjdndikokhpoecoloebgeibde/content_scripts/copy.js"></script>
 ▶ <head> ··· </head>
 ▶ <body class="home light"> ··· </body> == $0
</html>
```

4.3 小结

项目	结论
防御措施	无
绕过难度	无需绕过
可用 Payload	任意 XSS 向量均可直接执行

五、Medium 等级测试

5.1 源码分析

服务端 (PHP):

```
<?php
if ( array_key_exists( "default", $_GET ) && !is_null ($_GET[ 'default' ]) ) {
$default = $_GET['default'];

# Do not allow script tags
if (stripos ($default, "<script") !== false) {
header ("location: ?default=English");
exit;
}
}
?>
```

安全措施	分析
stripos()检测	大小写不敏感检测<script 字符串，匹配则重定向
覆盖范围	**仅防<script>标签**，其他标签不受限
前端漏洞	document.write()仍然存在，未修复

与反射型 XSS 的关键区别:stripos()是大小写不敏感的，因此大小写混用绕过在此无效。但其他 HTML 标签 + 事件处理器仍可绕过。

5.2 测试步骤

Step 1: 将 Security Level 设置为 Medium

Step 2: 验证<script>标签被拦截

?default=<script>alert('XSS')</script>

预期结果: 页面被重定向到?default=English，注入失败。

?default=<Script>alert('XSS')</Script>

预期结果: 同样被重定向（stripos 大小写不敏感）。

Step 3: 绕过方法 — 使用<img>标签

由于注入点在<select><option>内部，需要先闭合</select>标签，否则<img>等标签会被<select>吞掉：

?default=</select><img src=1 onerror=alert('XSS')>

原理:

1. </select>闭合了原有的下拉框标签
2. <img src=1 onerror=alert('XSS')>作为独立元素被写入页面
3. 图片加载失败触发 onerror 事件，执行脚本

预期结果: 弹窗成功。

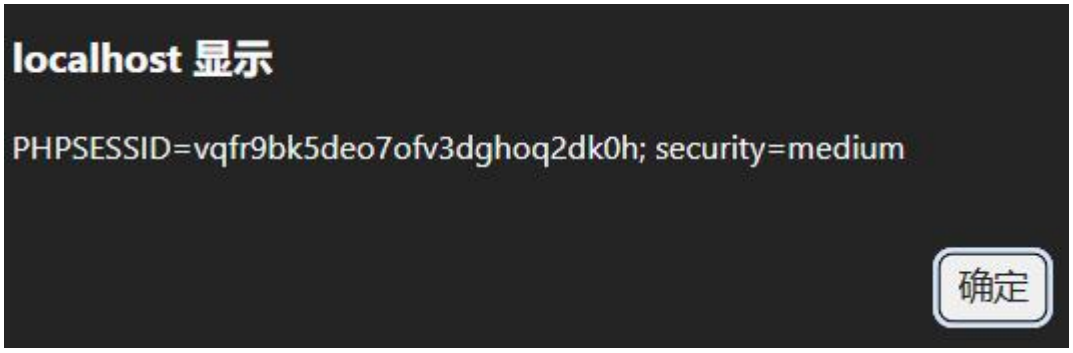
Step 4: 其他绕过 Payload

?default=</select><svg onload=alert('XSS')>
?default=</select><body onload=alert('XSS')>
?default=</select><input onfocus=alert('XSS') autofocus>

Step 5: 窃取 Cookie Payload

?default=</select><img src=1 onerror=alert(document.cookie)>

预期结果: 弹窗显示当前用户 Cookie。



5.3 小结

项目	结论
防御措施	stripos()检测<script（大小写不敏感）
绕过难度	低
关键技巧	需先用</select>闭合下拉框标签
有效绕过方法	</select>+ 替代标签（img/svg/body/input）

六、High 等级测试

6.1 源码分析

服务端 (PHP):

```
<?php
if ( array_key_exists( "default", $_GET ) && !is_null ($_GET[ 'default' ]) ) {

# White list the allowable languages
switch ($_GET['default']) {
case "French":
case "English":
case "German":
case "Spanish":
# ok
break;
default:
header ("location: ?default=English");
exit;
}
}
?>
```

安全措施	分析
白名单验证	switch/case 仅允许 4 种语言值，其他一律重定向
防御强度	**服务端层面非常强—** 黑名单无法绕过
致命盲区	**URL 片段 (#) 不会发送到服务器**

6.2 核心绕过原理

这是 DOM-XSS 最经典的攻击手法 **—利用 URL 片段标识符 (#)**:

http://target/vulnerabilities/xss_d/?default=English#<script>alert('XSS')</script>

为什么能绕过白名单?

完整 URL: ?default=English#<script>alert('XSS')</script>

服务端收到的部分 服务端收不到的部分
($_GET['default'] (#后的内容永远不会将= "English" ✓) 发送到服务器)

1. #及其后面的内容是 URL 片段标识符（fragment），**浏览器不会将其发送到服务端**
2. 服务端$_GET['default']得到的值是 English，通过白名单验证
3. 但前端 JavaScript 的 document.location.href 读取的是**完整 URL**（包含#部分）
4. document.location.href.substring(...) 提取 default= 后的所有内容，得到 English#<script>alert('XSS')</script>
5. document.write()将其写入页面，<script>标签被浏览器解析执行

6.3 测试步骤

Step 1: 将 Security Level 设置为 High

Step 2: 验证白名单有效 — 常规注入均被拦截

?default=<script>alert('XSS')</script> ← 重定向
?default=</select><img src=1 onerror=alert(1)> ← 重定向
?default=test ← 重定向

以上全部被重定向到?default=English，白名单生效。

Step 3: 使用#片段绕过

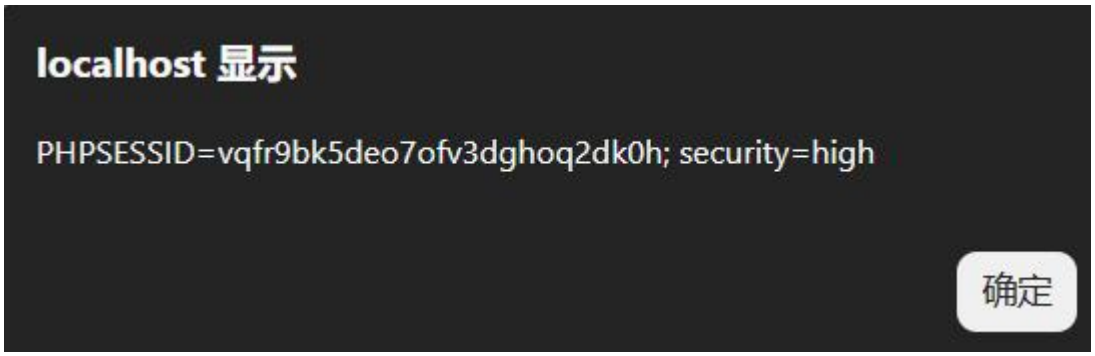
?default=English#<script>alert('XSS')</script>

预期结果: 服务端白名单通过（只看到 English），前端 JavaScript 读到完整 URL 并将<script>写入 DOM，弹窗成功。

Step 4: 窃取 Cookie Payload

?default=English#<script>alert(document.cookie)</script>

预期结果: 弹窗显示当前用户 Cookie。



Step 5: 验证服务端日志无记录

查看 Apache/Nginx 访问日志，确认#后的内容**不出现在日志中**：

日志记录: GET /vulnerabilities/xss_d/?default=English HTTP/1.1
实际攻击: ?default=English#<script>alert(document.cookie)</script>
↑ 此处之后的内容日志中完全不可见

6.4 小结

项目	结论
防御措施	服务端 switch/case 白名单（极强）
绕过难度	中
核心绕过原理	#片段不发送到服务器，但 JS 可读取
有效 Payload	English#<script>alert(document.cookie)</script>
隐蔽性	**极高**— 服务端日志无任何异常记录

七、三等级横向对比

维度	Low	Medium	High
服务端防御	无	stripos 检测<script	switch/case 白名单
前端防御	无	无	无
核心绕过思路	直接注入	闭合</select>+ 替代标签	#URL 片段绕过
绕过难度	无	低	中
服务端可检测	是	是	**否（#不上传）**

根本问题：前端 document.write()直接输出不可信数据。
核心结论: DOM-XSS 的根本漏洞在前端 JavaScript 代码，三个等级的服务端防护逐步加强，

但均未修复前端的 document.write()问题。High 等级利用#片段特性实现了**完全绕过服务端检测**的攻击，体现了 DOM-XSS "无服务端痕迹"的核心威胁。

八、修复建议与代码示例

8.1 正确的修复方案（对应 DVWA Impossible 等级思路）

Impossible 等级服务端代码:

```
<?php
# Don't need to do anything, protection handled on the client side
?>
```

Impossible 等级前端关键变更:

```
// Impossible 等级去掉了 decodeURI，保留 URL 编码
document.write("<option value='" + lang + "'>" + (lang) + "</option>");
// ^^^^
// 不再调用 decodeURI()
// %3Cscript%3E 保持编码状态，不会被解析为标签
```

8.2 更彻底的修复方案

```
// 方案一：使用 DOM API 代替 document.write()
if (document.location.href.indexOf("default=") >= 0) {
var lang = document.location.href.substring(
document.location.href.indexOf("default=") + 8
);
var select = document.getElementsByName("default")[0];
var option = document.createElement("option");
option.value = lang;
option.textContent = decodeURI(lang); // textContent 自动转义
select.appendChild(option);
}

// 方案二：使用白名单验证
var allowedLangs = ["English", "French", "Spanish", "German"];
```

```
if (allowedLangs.indexOf(decodeURI(lang)) >= 0) {
document.write("" + decodeURI(lang) + "");
}
```

8.3 修复要点

修复项	说明
避免 document.write()	使用 textContent/createElement 等安全 DOM API
不使用 innerHTML	innerHTML 与 document.write 同样危险
前端白名单验证	在 JavaScript 中也对输入值做白名单校验
去除 decodeURI()	阻止 URL 编码被还原为可执行的 HTML
CSP 策略	Content-Security-Policy: script-src 'self'阻止内联脚本

8.4 防御体系建议

层级	措施	说明
前端代码层	textContent 代替 innerHTML/document.write	根本解决 DOM-XSS
前端验证层	JavaScript 白名单校验输入	防止非法值进入 DOM
HTTP 层	Content-Security-Policy	禁止内联脚本执行
开发规范	禁止使用 eval(),document.write(),innerHTML 等危险 API	从编码规范层面杜绝
测试层	基于 DOM 的自动化 XSS 扫描工具	传统扫描器可能漏报 DOM-XSS

九、训练总结

本次基于 DVWA DOM Based XSS 模块的训练覆盖了三个安全等级的攻防对抗：

1. **Low**: 理解 DOM-XSS 的基本原理 — JavaScript 从 URL 读取输入并通过 document.write()写入页面
2. **Medium**: 理解闭合标签技巧 — 需要先用</select>闭合父级标签，再注入替代标签
3. **High**: 理解#URL 片段的核心特性 — 片段不发送到服务器，可完全绕过服务端白名单

关键训练收获:

- DOM-XSS 的根因在**前端 JavaScript**，服务端防护无论多强都无法根治
- #片段攻击使 DOM-XSS 具有**零服务端痕迹**的特性，传统安全监控完全失效

- document.write()和 innerHTML 是前端最危险的 API，应在编码规范中明确禁止
- DOM-XSS 是三种 XSS 中**最难检测**的类型，需要专门的前端安全审计